

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

И.Д. Свеженин

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Введение в мобильную разработку, Первое
приложение в Android Studio;

по курсу: Кроссплатформенное программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1. Задание.....	2
2. Ход работы	3
3. Результат работы программы	6
4. Вывод	8
5. Листинг с кодом программы	9

1. Задание

Задание данной лабораторной работы заключалось в успешной установке среды разработки Android Studio, изучении основных принципов и базовых механизмов мобильной разработки под операционную систему Android, а также в создании собственного приложения — инженерного калькулятора. В процессе выполнения работы необходимо было ознакомиться с интерфейсом Android Studio, структурой проекта и файлами, отвечающими за визуальную часть и логику приложения. Основной целью являлось закрепление навыков работы с XML-разметкой и языком программирования Kotlin, а также получение практического опыта создания пользовательского интерфейса, настройки элементов управления и обработки событий нажатия. Итогом выполнения задания стало разработанное мобильное приложение инженерного калькулятора с базовыми функциями вычислений и удобным пользовательским интерфейсом.

2. Ход работы

Ход работы начался с создания нового проекта в Android Studio с использованием языка программирования Kotlin и шаблона Empty Activity. В первую очередь была выполнена настройка интерфейса в файле `activity_main.xml`, где определялась основная структура пользовательского окна калькулятора. Для удобства восприятия и корректного отображения приложения на различных устройствах был установлен вертикальный режим ориентации экрана. Основным контейнером интерфейса выбран элемент `LinearLayout` с вертикальной ориентацией, внутри которого размещались все составляющие — поле вывода данных и блоки кнопок. В верхней части интерфейса было добавлено текстовое поле `TextView` для отображения вводимых чисел и результатов вычислений, которому были заданы отступы, размеры и параметры выравнивания, обеспечивающие удобное восприятие информации пользователем.

Затем были настроены визуальные параметры приложения в файле `colors.xml`, где определялись основные цвета интерфейса: цвет фона, цвет текста и оттенки для активных и неактивных кнопок. Это позволило создать единый дизайн в спокойной цветовой гамме, обеспечивающей комфорт при длительном использовании. Далее в файле `styles.xml` были оформлены шаблоны стилей для двух типов элементов: кнопок с цифрами и кнопок операций (таких как сложение, вычитание, деление и умножение). Для цифровых кнопок применялись базовые параметры выравнивания, размера текста и цвета фона, а для кнопок действий — выделенные цвета и тени, подчеркивающие их активность. Такой подход обеспечил единообразный и аккуратный внешний вид всех элементов интерфейса.

После завершения оформления интерфейса началась работа с логикой приложения в файле `MainActivity.kt`. В этом классе создавалась основная функциональность инженерного калькулятора. В начале работы осуществляется связывание элементов интерфейса с кодом при помощи механизма View Binding (через объект `ActivityMainBinding`), что упрощает

обращение к элементам разметки без необходимости использовать `findViewById`. Далее определяются несколько вспомогательных переменных: `canAddOperation` и `canAddDecimal` — для контроля ввода операций и десятичных чисел, `memoryValue` — для хранения значения в памяти калькулятора, а также `isDegreeMode`, который отвечает за переключение между градусами и радианами при вычислении тригонометрических функций.

Основные функции интерфейса обеспечиваются методами-обработчиками событий для каждой группы кнопок. Методы `numberAction` и `operationAction` реализуют ввод чисел и математических операций, при этом контролируется возможность добавления точки и последовательности операторов. Методы `allClearAction` и `backSpaceAction` очищают поле ввода или удаляют последний символ, а `equalsAction` вызывает функцию `calculateResults`, где происходит вычисление выражения. Дополнительно реализованы функции работы с памятью (`memoryPlusAction`, `memoryMinusAction`, `memoryRecallAction`), позволяющие сохранять промежуточные результаты, а также функции для изменения знака числа, вычисления факториала, процента, возведения в степень и переключения режима радианы/градусы.

Вычислительная логика сосредоточена в методе `calculateResults`, где используется сторонняя библиотека `exp4j` для безопасного и удобного парсинга математических выражений. Перед вычислением ввод пользователя преобразуется: символы “х”, “÷”, “√”, “π” и другие заменяются на корректные обозначения, понятные библиотеке. Также с помощью регулярных выражений обрабатываются операции факториала, процента и обратного значения. Далее создаются собственные функции (`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `log`, `ln`, `exp`, `sqrt`), в которых учитывается текущий режим измерения углов — в градусах или радианах. После этого с помощью класса `ExpressionBuilder` формируется выражение и производится вычисление результата. Полученное значение форматируется: если результат является целым числом, выводится без дробной части, а в противном случае отображается с ограниченной точностью. При ошибках ввода или вычислений выводится сообщение “Ошибка”. Таким

образом, данный файл реализует как логику взаимодействия с пользователем, так и весь процесс обработки и вычисления математических выражений, обеспечивая корректную работу инженерного калькулятора с поддержкой тригонометрических, логарифмических и экспоненциальных функций.

3. Результат работы программы

В качестве результата работы программы представлен вывод значений в терминал. На рисунках 1-2 показан пример работы программы с различными значениями.

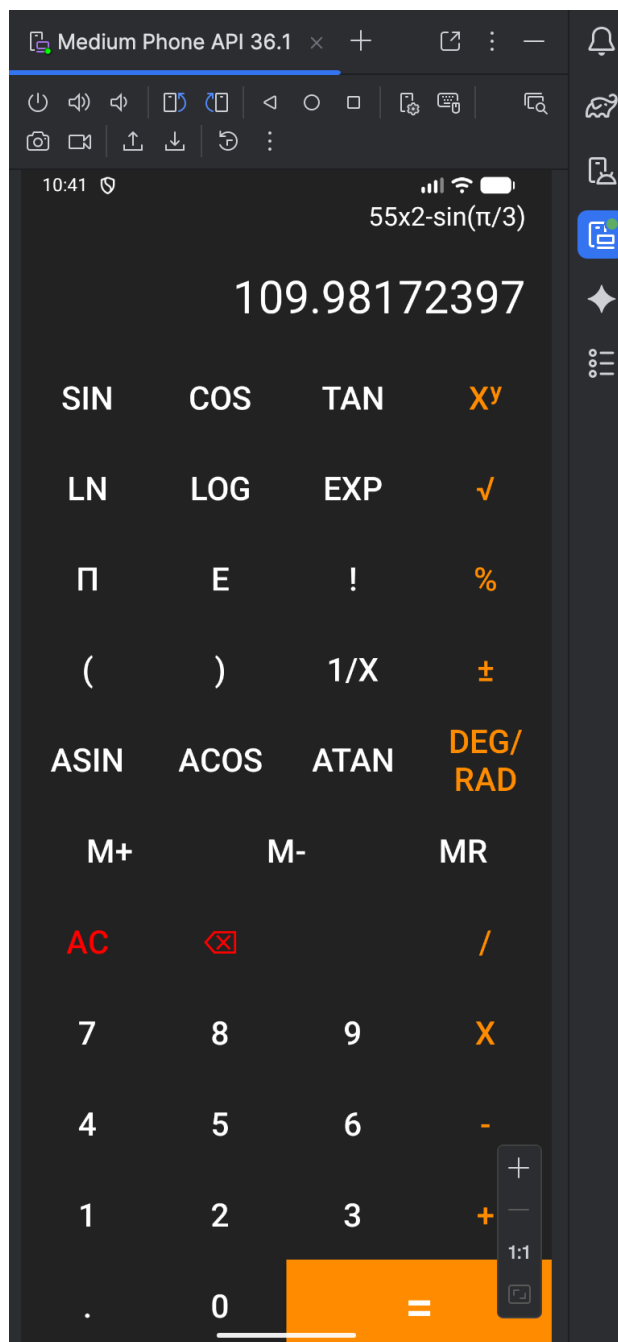


Рисунок 1 – Успешный подсчет с различными математическими функциями

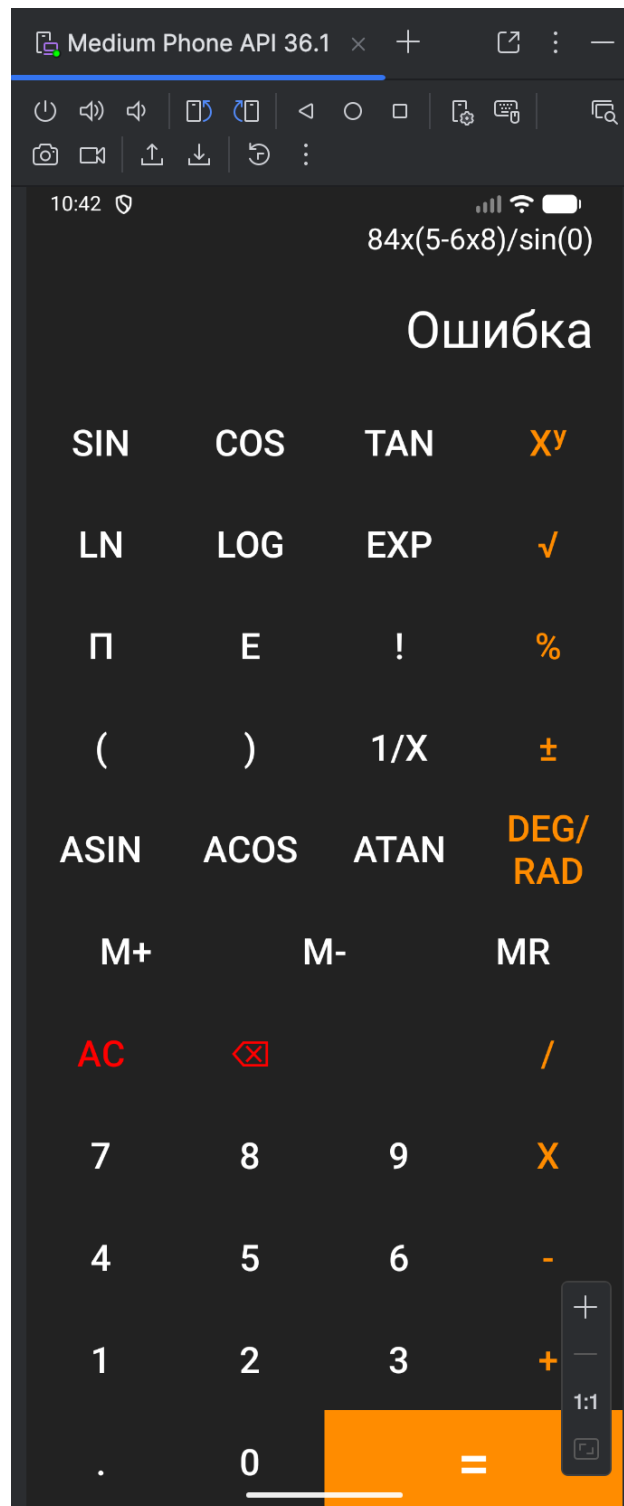


Рисунок 2 – Успешно обработанная ошибка

4. Вывод

В ходе выполнения лабораторной работы было успешно выполнено полное развертывание среды Android Studio, изучены основные механизмы построения мобильных приложений и реализовано собственное приложение — инженерный калькулятор. В процессе разработки были освоены базовые принципы работы с XML-разметкой, настройкой стилей и цветовых схем, а также создание интерфейсов с использованием современных инструментов Android-разработки. Отдельное внимание было уделено организации структуры проекта, оформлению элементов интерфейса и связке визуальной части с кодом при помощи механизма View Binding, что повысило удобство и читаемость программы.

Финальным результатом работы стало полностью функционирующее мобильное приложение, способное выполнять как простые арифметические, так и сложные инженерные вычисления, включая работу с тригонометрическими, логарифмическими и экспоненциальными функциями. Благодаря использованию библиотеки `exp4j` обеспечена корректная обработка математических выражений и высокая точность вычислений. Реализованный калькулятор обладает удобным интерфейсом, устойчив к ошибкам ввода и поддерживает различные режимы вычислений, что подтверждает успешное достижение всех поставленных целей лабораторной работы и закрепление навыков разработки приложений под Android.

5. Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
//MainActivity

package com.example.engineercalc

import android.os.Bundle
import android.view.View
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
import com.example.engineercalc.databinding.ActivityMainBinding
import net.objecthunter.exp4j.ExpressionBuilder
import net.objecthunter.exp4j.function.Function

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private var canAddOperation = false
    private var canAddDecimal = true
    private var memoryValue = 0.0
    private var isDegreeMode = true

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
    }

    fun degRadAction(view: View) {
        isDegreeMode = !isDegreeMode
        binding.resultsTV.text = if (isDegreeMode) "DEG" else "RAD"
    }
}
```

```
}
```

```
fun numberAction(view: View) {  
    if (view is Button) {  
        val text = view.text.toString()  
        if (text == ".") {  
            if (canAddDecimal) {  
                binding.workingsTV.append(".")  
                canAddDecimal = false  
            }  
        } else {  
            binding.workingsTV.append(text)  
        }  
        canAddOperation = true  
    }  
}
```

```
fun operationAction(view: View) {  
    if (view is Button && canAddOperation) {  
        binding.workingsTV.append(view.text)  
        canAddOperation = false  
        canAddDecimal = true  
    }  
}
```

```
fun allClearAction(view: View) {  
    binding.workingsTV.text = ""  
    binding.resultsTV.text = ""  
    canAddDecimal = true  
}
```

```

fun backSpaceAction(view: View) {
    val text = binding.workingsTV.text.toString()
    if (text.isNotEmpty()) {
        binding.workingsTV.text = text.dropLast(1)
    }
}

```

```

fun equalsAction(view: View) {
    binding.resultsTV.text = calculateResults()
}

```

```

fun functionAction(view: View) {
    if (view is Button) {
        val text = view.text.toString()
        binding.workingsTV.append("$text(")
        canAddOperation = false
    }
}

```

```

fun parenthesesAction(view: View) {
    if (view is Button) {
        binding.workingsTV.append(view.text)
    }
}

```

```

fun constantAction(view: View) {
    if (view is Button) {
        binding.workingsTV.append(view.text)
        canAddOperation = true
    }
}

```

```

    }
}

```

```

fun toggleSignAction(view: View) {
    val expr = binding.workingsTV.text.toString()
    if (expr.isEmpty()) return
    val regex = Regex("(-?\\d+(\\.\\d+)?)$")
    val match = regex.find(expr)
    if (match != null) {
        val number = match.value.toDouble()
        val newExpr = expr.replaceRange(match.range, (-number).toString())
        binding.workingsTV.text = newExpr
    }
}

```

```

fun factAction(view: View) {
    binding.workingsTV.append("!")
}

```

```

fun reciprocalAction(view: View) {
    val expr = binding.workingsTV.text.toString()
    if (expr.isNotEmpty()) {
        binding.workingsTV.text = "1/($expr)"
    }
}

```

```

fun percentAction(view: View) {
    val expr = binding.workingsTV.text.toString()
    if (expr.isNotEmpty()) {
        binding.workingsTV.append("%")
    }
}

```

```

    }
}

fun powerAction(view: View) {
    binding.workingsTV.append("^")
}

fun memoryPlusAction(view: View) {
    val result = calculateResults().toDoubleOrNull()
    if (result != null) {
        memoryValue += result
    }
}

fun memoryMinusAction(view: View) {
    val result = calculateResults().toDoubleOrNull()
    if (result != null) {
        memoryValue -= result
    }
}

fun memoryRecallAction(view: View) {
    binding.workingsTV.append(
        if (memoryValue % 1 == 0.0) memoryValue.toInt().toString()
        else memoryValue.toString()
    )
}

private fun calculateResults(): String {
    return try {

```

```

var expressionText = binding.workingsTV.text.toString()
    .replace("x", "*")
    .replace("÷", "/")
    .replace("√", "sqrt")
    .replace("π", Math.PI.toString())
    .replace("e", Math.E.toString())
    .replace("xy", "^")

expressionText = expressionText.replace(Regex("(\\d+)!")) { match ->
    val num = match.groupValues[1].toInt()
    val fact = if (num >= 0) (1..num).fold(1.0) { acc, i -> acc * i } else
Double.NaN
    fact.toString()
}

expressionText = expressionText.replace(Regex("(\\d+(\\.\\d+)?)%")) {
    "(${it.groupValues[1]}/100)"
}

expressionText = expressionText.replace(Regex("1\\^\\([\\^]+\\)\\\\")) {
    val inner = it.groupValues[1]
    "(1/($inner))"
}

val sin = object : Function("sin", 1) {
    override fun apply(vararg args: Double): Double {
        return if (isDegreeMode) Math.sin(Math.toRadians(args[0])) else
Math.sin(args[0])
    }
}

```

```

val cos = object : Function("cos", 1) {
    override fun apply(vararg args: Double): Double {
        return if (isDegreeMode) Math.cos(Math.toRadians(args[0])) else
Math.cos(args[0])
    }
}

```

```

val tan = object : Function("tan", 1) {
    override fun apply(vararg args: Double): Double {
        return if (isDegreeMode) Math.tan(Math.toRadians(args[0])) else
Math.tan(args[0])
    }
}

```

```

val asin = object : Function("asin", 1) {
    override fun apply(vararg args: Double): Double {
        val value = Math.asin(args[0])
        return if (isDegreeMode) Math.toDegrees(value) else value
    }
}

```

```

val acos = object : Function("acos", 1) {
    override fun apply(vararg args: Double): Double {
        val value = Math.acos(args[0])
        return if (isDegreeMode) Math.toDegrees(value) else value
    }
}

```

```

val atan = object : Function("atan", 1) {

```

```

        override fun apply(vararg args: Double): Double {
            val value = Math.atan(args[0])
            return if (isDegreeMode) Math.toDegrees(value) else value
        }
    }

    val log = object : Function("log", 1) {
        override fun apply(vararg args: Double): Double = Math.log10(args[0])
    }

    val ln = object : Function("ln", 1) {
        override fun apply(vararg args: Double): Double = Math.log(args[0])
    }

    val exp = object : Function("exp", 1) {
        override fun apply(vararg args: Double): Double = Math.exp(args[0])
    }

    val sqrt = object : Function("sqrt", 1) {
        override fun apply(vararg args: Double): Double = Math.sqrt(args[0])
    }

    val expression = ExpressionBuilder(expressionText)
        .function(sin)
        .function(cos)
        .function(tan)
        .function(asin)
        .function(acos)
        .function(atan)
        .function(log)

```

```

        .function(ln)
        .function(exp)
        .function(sqrt)
        .build()

val result = expression.evaluate()

if (result.isNaN() || result.isInfinite()) {
    "Ошибка"
} else if (result % 1 == 0.0) {
    result.toInt().toString()
} else {
    String.format(java.util.Locale.US, "%.8f", result)
        .trimEnd('0')
        .trimEnd('.')
    }
} catch (e: Exception) {
    "Ошибка"
}
}
}

```

//activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"

```

```
android:layout_height="match_parent"
android:orientation="vertical"
android:background="@color/almostBlack"
tools:ignore="HardcodedText"
tools:context=".MainActivity">
```

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="2"
    android:padding="20dp">
```

```
<TextView
    android:id="@+id/workingsTV"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text=""
    android:lines="2"
    android:maxLength="2"
    android:textColor="@color/white"
    android:textAlignment="textEnd"
    android:textSize="20sp"
    app:layout_constraintBottom_toTopOf="@id/resultsTV"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

```
<TextView
    android:id="@+id/resultsTV"
    android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
android:text=""
android:lines="1"
android:maxLines="1"
android:textColor="@color/white"
android:textAlignment="textEnd"
android:textSize="35sp"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintLeft_toLeftOf="parent"
app:layout_constraintRight_toRightOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

```
<LinearLayout style="@style/buttonRow">
```

```
<Button
    style="@style/buttonNumber"
    android:text="sin"
    android:onClick="functionAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="cos"
    android:onClick="functionAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="tan"
    android:onClick="functionAction" />
```

```

<Button
    style="@style/buttonOperator"
    android:text="xy"
    android:onClick="powerAction" />
</LinearLayout>

<LinearLayout style="@style/buttonRow">
    <Button
        style="@style/buttonNumber"
        android:text="ln"
        android:onClick="functionAction" />

    <Button
        style="@style/buttonNumber"
        android:text="log"
        android:onClick="functionAction" />

    <Button
        style="@style/buttonNumber"
        android:text="exp"
        android:onClick="functionAction" />

    <Button
        style="@style/buttonOperator"
        android:text="√"
        android:onClick="functionAction" />
</LinearLayout>

<LinearLayout style="@style/buttonRow">
    <Button

```

```
style="@style/buttonNumber"
android:text="π"
android:onClick="constantAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="e"
    android:onClick="constantAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="!"
    android:onClick="factAction" />
```

```
<Button
    style="@style/buttonOperator"
    android:text="%"
    android:onClick="percentAction" />
```

```
</LinearLayout>
```

```
<LinearLayout style="@style/buttonRow">
```

```
<Button
    style="@style/buttonNumber"
    android:text="("
    android:onClick="parenthesesAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text=")"
    android:onClick="parenthesesAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="1/x"
    android:onClick="reciprocalAction" />
```

```
<Button
    style="@style/buttonOperator"
    android:text="±"
    android:onClick="toggleSignAction" />
```

```
</LinearLayout>
```

```
<LinearLayout style="@style/buttonRow">
```

```
<Button
    style="@style/buttonNumber"
    android:text="asin"
    android:onClick="functionAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="acos"
    android:onClick="functionAction" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="atan"
    android:onClick="functionAction" />
```

```
<Button
    style="@style/buttonOperator"
```

```

        android:text="DEG/RAD"
        android:onClick="degRadAction" />
</LinearLayout>

<LinearLayout style="@style/buttonRow">
    <Button
        style="@style/buttonNumber"
        android:text="M+"
        android:onClick="memoryPlusAction" />

    <Button
        style="@style/buttonNumber"
        android:text="M-"
        android:onClick="memoryMinusAction" />

    <Button
        style="@style/buttonNumber"
        android:text="MR"
        android:onClick="memoryRecallAction" />
</LinearLayout>

<LinearLayout style="@style/buttonRow">

    <Button
        style="@style/buttonNumber"
        android:textColor="@color/red"
        android:onClick="allClearAction"
        android:text="AC" />

```

```
<Button
    style="@style/buttonNumber"
    android:textColor="@color/red"
    android:onClick="backSpaceAction"
    android:text="⊠"
/>
```

```
<Button
    style="@style/buttonNumber"
    android:text=" " />
```

```
<Button
    style="@style/buttonOperator"
    android:text="/" />
```

```
</LinearLayout>
```

```
<LinearLayout style="@style/buttonRow">
```

```
<Button
    style="@style/buttonNumber"
    android:text="7" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="8" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="9" />
```

```
<Button
    style="@style/buttonOperator"
    android:text="x" />
```

```
</LinearLayout>
```

```
<LinearLayout style="@style/buttonRow">
```

```
<Button
    style="@style/buttonNumber"
    android:text="4" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="5" />
```

```
<Button
    style="@style/buttonNumber"
    android:text="6" />
```

```
<Button
    style="@style/buttonOperator"
    android:text="-" />
```

```
</LinearLayout>
```

```
<LinearLayout style="@style/buttonRow">
```

```
<Button  
    style="@style/buttonNumber"  
    android:text="1" />
```

```
<Button  
    style="@style/buttonNumber"  
    android:text="2" />
```

```
<Button  
    style="@style/buttonNumber"  
    android:text="3" />
```

```
<Button  
    style="@style/buttonOperator"  
    android:text="+" />
```

```
</LinearLayout>
```

```
<LinearLayout style="@style/buttonRow">
```

```
<Button  
    style="@style/buttonNumber"  
    android:text="." />
```

```
<Button  
    style="@style/buttonNumber"  
    android:text="0" />
```

```
<Button
    style="@style/buttonNumber"
    android:background="@color/orange"
    android:layout_weight="2"
    android:textSize="40sp"
    android:onClick="equalsAction"
    android:text="" />
```

```
</LinearLayout>
```

```
</LinearLayout>
```